

Introduction

After understanding the PC Timer/Counter, we will delve into one of the most critical I/O Devices capable of enabling the communication with a traditional computer, the **PC Keyboard**.

The keyboard is essential since it is one of the primary ways users **communicate instructions, data, and commands to the system**. From logging in and writing documents to programming, gaming, and system control, the keyboard enables precise, fast, and reliable input. Unlike other I/O devices, the PC keyboard allow users to interact with a PC at a very low level, including during startup, troubleshooting, and situations where other input methods are unavailable.

Even modern graphical systems still **depend** on keyboards for shortcuts, accessibility, and productivity. Operating systems, programming environments, and professional applications are all designed with keyboard input in mind, making it an **essential PC component**.

Lab3: Core objectives

- Understand how the PC Keyboard operates and how to use its low level interface
- Discuss the main advantages and disadvantages of **Interrupts vs Polling**
- Be able to handle multiple I/O Devices: The PC Timer/Counter and the PC Keyboard

Plan

In the first class, you are expected to understand the communication mechanisms between the PC and the PC Keyboard and how to interact with the PC Keyboard controller. This is crucial to be able to react to Keyboard inputs, either the press or release of keys, generating a makecode or a breakcode, respectively. Therefore, you are required to develop two functions that read and display all scancodes generated by the PC keyboard where the first takes advantage of keyboard interrupts, and the second employs polling mechanisms.

In the second class, you will develop a function that handles two I/O devices: the PC Timer/Counter and the PC keyboard. Similarly to the first exercise of this lab, you will read and display keyboard scancodes; however, it will additionally use the timer to control the function's lifetime.

We advise you to read everything carefully, refrain from a blind use of AI, and use the support of the teaching assistants whenever you feel stuck. Good luck and have fun!

References

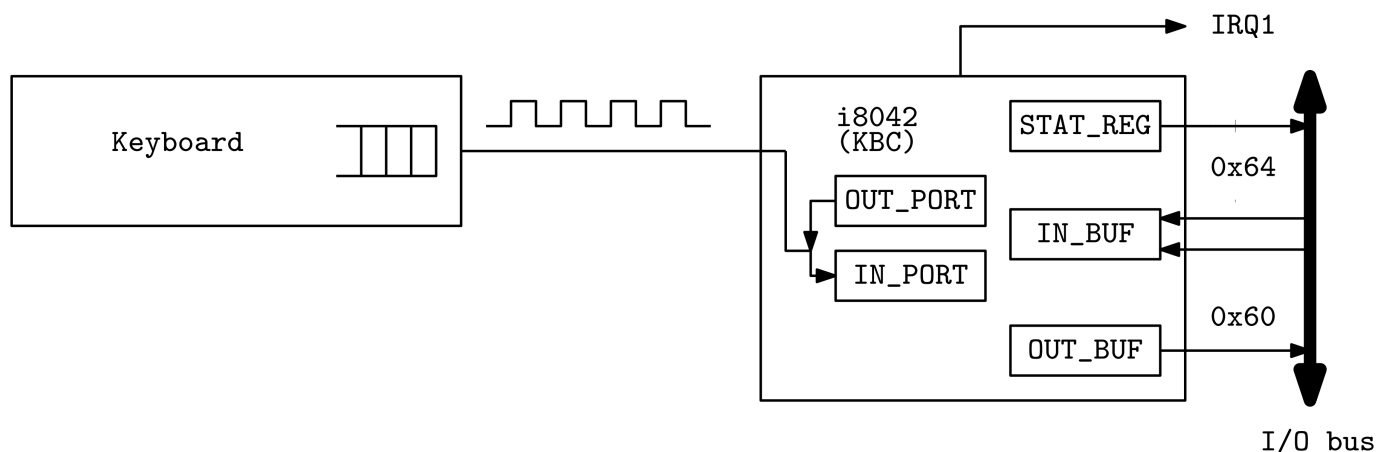
Here you have some relevant files and documentation for the PC Keyboard lab:

- [lab3.c](#) file that contains the test function stubs that you will develop
 - [Doxygen documentation](#)
 - [8042 functional description from the IBM Technical Reference Manual](#)
 - [Data sheet of an 8042-compatible IC](#)
-

The PC Keyboard

This chapter explores the communication process between the Keyboard (**KBD**) and the PC, detailing how input signals generated by keypresses are transmitted and managed to enable effective interaction between the user and the system.

The following image represents the Keyboard, transmitting data to the Keyboard Controller (**KBC**).



The Controller at the Keyboard: C@KBD

A keyboard reacts to key presses and releases. Most keyboards have their own standalone controller chip, the **Controller at the Keyboard (C@KBD)**, which is responsible for encoding key events. Depending on the interface (PS/2, USB, Bluetooth), the keyboard sends either scancodes or protocol-specific key reports to the host.

The conversion between the key's scancodes and the code of the character on that key is usually done by the operating system using a keymap. For example, by default, Minix 3 uses the US keyboard keymap. The Minix 3 image you are using in LCOM uses a Portuguese keymap. The advantage of this approach for keyboard manufacturers is clear: a key in a given position, always generates the same scancode, independently of whether the keyboard uses the US layout or another layout.

To transmit all generated scancodes, the **C@KBD** communicates with the PC via **serial line**, i.e., the bits in a byte are sent one after the other over wires.

Scancode

A Scancode is a **one or two byte-long** code associated with a specific keyboard key. A key's **scancode** depends only on the **position** of that key in the keyboard and do **not** follow the

ASCII code or other common encoding. The conversion between the key's scancodes and the code of the character on that key is usually done by the OS using a **keymap**.

The advantage of this approach for keyboard manufacturers is clear: a key in a given position **always generates the same scancode**, independently of whether the keyboard uses the Portuguese layout (**default layout for LCOM-MINIX**), US layout (**default layout for MINIX 3**), or another layout.

- **Makecode**: scancode generated when the key is **pressed**
- **Breakcode**: scancode generated when the key is **released**

Important

Usually the **Breakcode** generated when a key is released is the **Makecode** of that key with the **MSB** set to **1**

Example: the makecode of the **ESC** key is **0x01** whereas its breakcode is **0x81**.

Scancode Specification

Most PC's scancodes are a **single byte long**, such as the ESC makecode (0x01) and breakcode (0x81). Nevertheless, some special keys have **two-byte long** scancodes which usually use **0xE0** as their first byte. This prefix is used in **both** the make and the break codes.

In case of two-byte scancodes, the **key** difference between the **makecode** and **breakcode** of a key is present in the **most significant bit (MSB)** of the second byte.

Example of 2-byte scancode

If the makecode is **0xE06A**, then the breakcode must be **0xE0EA**

The Keyboard Controller (KBC)

In modern PCs, the communication between the keyboard and the processor is mediated by an electronic component that provides the functionality of the **i8042**, the **Keyboard Controller (KBC)**. The **KBC** is responsible for the communication management on the PC side and, in modern PCs, is usually directly integrated in the motherboard's chipset.

KBC Registers

As illustrated in the previous figure, all communication with the Keyboard Controller (KBC) occurs through ports **0x60** and **0x64**, which are used for both **reading** from and **writing** to the controller.

Port	Operation	Name	Action
0x60	read	Output Buffer	Read scancodes or returned values from KBC commands
0x60	write	Input Buffer	Write arguments of KBC commands
0x64	read	Status Register	Read the KBC status
0x64	write	Command Register	Write KBC commands

KBC Status Register

Reading the **KBC Status Register** is crucial because it provides essential information about the controller's current state before any data transfer is attempted. The **status register Byte** indicates whether the input or output buffers are full, allowing the system to determine when it is safe to write commands or read data without causing data loss or corruption. It also reports error conditions, such as parity or timeout errors, enabling reliable detection and handling of communication faults. By consulting the status register, the system ensures synchronized and robust communication with the KBC.

The following table specifies the **Status Register Byte** format:

Bit	Name	Meaning (If set to 1)
7	Parity	Parity error - invalid data
6	Timeout	Timeout error - invalid data
5	Aux	Mouse data
4	INH	Inhibit flag: 0 if keyboard is inhibited
3	A2	A2 input line: irrelevant for LCOM
2	SYS	System flag: irrelevant for LCOM
1	IBF	Input buffer full: don't write commands or arguments
0	OBF	Output buffer full: data available for reading

 Important

- Bits 7 and 6 should **always** be checked in the **Interrupt Handler** since they signal communication errors between the keyboard and the KBC
- Even if an error occur, **the Output Buffer must always be read but the value must be discarded**

From the C@KBD to generating an interrupt

Finally, you should keep in mind the sequence of events that occurs when a keyboard event is communicated to the PC.

1. When a key is pressed or released, the **C@KBD** generates the corresponding **scancode** and **stores it** temporarily in its buffer
2. If the **KBC Output Buffer is empty**, the **C@KBD** transmits the scancode at the head of its buffer to the **KBC**
3. The **KBC** then places this scancode into its **Output Buffer**, making it accessible
4. Finally, the **KBC** signals the arrival of new keyboard data to the CPU by **raising interrupt IRQ1**
5. After **reading** the **scancode** in the **Interrupt Handler**, **KBC** signals the **C@KBD** via serial bus that the Output Buffer is now empty

Next Steps

Now that you have learned about the PC Keyboard and its controllers, you are ready to complete your first task: [Reading Scancodes from KBC](#).

Reading Scancodes from KBC

Now that you are familiar with the Keyboard Controller, it is time to put your hands dirty. The first proposed exercise is for you to implement the `kbd_test_scan()` function using **LCF** with the purpose of correctly reading the scancodes from the KBC using an Interrupt Handler (IH).

Tip

To help you, we have prepared the [lab3.c](#) file, which contains the stubs of the main functions to implement in this lab, together with [doxygen's documentation](#) of these functions and other useful utilities.

The `kbd_test_scan()` function

In the PC Keyboard lab, you will start by developing the `int kbd_test_scan()` function, which must read correctly all scancodes from the KBC. Since we are using interrupts, you will have to implement the interrupt handler `void kbc_ih()`, in which every time a byte from the keyboard is received, `kbd_test_scan()` should try to assemble a scancode. Furthermore, if there are no errors in KBC, you should print the scancode on the console by calling the following provided function:

```
#include <lcom/lcf.h>

int kbd_print_scancode(bool make, uint8_t size, uint8_t bytes[]);
```

Where:

- **make:** indicates whether this is a makecode or a breakcode
- **size:** number of bytes of the scancode
- **bytes:** is an array containing the bytes of the scancode read from the KBC. The array should contain the bytes **in the order they are read** from the KBC.

Note

Be aware that this function only accepts scancodes with 1 or 2 bytes, not supporting scancodes with 3 or more bytes (e.g., PAUSE, etc).

The output of this function will look like the following:

```

Makecode: 0x12
Breakcode: 0x92
Makecode: 0xe0 0x49
Breakcode: 0xe0 0xc9

```

Finally, this function must terminate when the breakcode of the ESC key, **0x81**, is detected.

Interrupts

Similar to what you have implemented in **lab2**, you will continue to take advantage of **interrupts**, in this case generated by the KBC, to handle the inputs of your **Keyboard**. However, instead of using the **IRQ 0**, which is associated with the Timer, the KBC raises interrupts to the **IRQ 1**. Additionally, you should still use the general structure of the main loop with all the functionalities already mentioned in the previous lab.

```

#include <lcom/lcf.h>

int ipc_status;
message msg;

while( 1 ) { /* You may want to use a different condition */
    /* Get a request message. */
    if ( (r = driver_receive(ANY, &msg, &ipc_status)) != 0 ) {
        printf("driver_receive failed with: %d", r);
        continue;
    }
    if (is_ipc_notify(ipc_status)) { /* received notification */
        switch (_ENDPOINT_P(msg.m_source)) {
            case HARDWARE: /* hardware interrupt notification */
                if (msg.m_notify.interrupts & irq_set) { /* subscribed
interrupt */
                    ... /* process it */
                }
                break;
            default:
                break; /* no other notifications expected: do nothing */
        }
    } else { /* received a standard message, not a notification */
        /* no standard messages expected: do nothing */
    }
}

```

Subscribing to KBC interrupts

Similarly to the last function of **lab2**, you must subscribe notifications from an IRQ line, now IRQ 1.

When MINIX boots, its keyboard driver configures the KBC and installs its own keyboard interrupt handler. This allows Minix to receive input from the keyboard of the PC (or virtual machine) on which it is running.

- On one hand this is **great**. You do not need to configure the KBC.
- On the other hand this **raises an issue**. How can your code read the scancodes?

Since you do not want to miss any input from the keyboard, you must subscribe interrupts exclusively so that the default keyboard driver process is not notified and does not read the data we are interested in reading.

Disabling MINIX Interrupt Handling

To address this issue, you should subscribe the KBC interrupts. However, the subscription must specify not only the **IRQ_REENABLE** policy but also the **IRQ_EXCLUSIVE** policy. As a result, the standard MINIX **KBC IH** will not be notified of the occurrence of KBC interrupts, thus preventing it from interfering with your code.

```
sys_irqsetpolicy(..., IRQ_REENABLE | IRQ_EXCLUSIVE, ...)
```

To ensure that you can use the Minix 3 terminal once your program is done, it must not forget to unsubscribe the KBC interrupts before exiting, by calling the `sys_irqrmpolicy()` kernel call.

Interrupt Handler

As already mentioned, the **IH** must be a function that takes no arguments and returns no value. Therefore, we recommend you to implement getter and setter functions to pass data between the **IH** and other functions. Although discouraged, an alternative solution is to use global variables.

In its definition, the function name of the `kbc_ih()` function must appear between parenthesis, as shown in the following code:

```
#include <lcom/lcf.h>

void (kbc_ih)() {
    // to be completed
}
```

The IH should perform two main tasks:

1. Read the **status register** and check if there was some communications error
2. Read and store a single **scancode** byte from the **KBC Output Buffer**



To follow a **layered implementation**, which will help you during the project development, you should implement this function in a **separate file**, e.g., `kbc.c`. You can follow the structure of Lab 2, where `timer.c` had the functions that interface with the i8254.

Makefile, Compiling and Running

To ease the development of this Lab, we should use a similar Makefile from Lab 2 to enable `make` commands. You can copy the previous **Makefile** and make the appropriate adjustments. Furthermore, you should add the following flag:

```
CFLAGS += -DLAB3
```

Then, you will be able to compile your code of Lab3 by executing the `make` command. After compiling, you can execute your program using the **lcom_run utility** as follows:

```
minix$ lcom_run lab3 "scan"
```

Testing with the LCF

Finally, you should test the `kbd_test_scan()` function using the **LCF testing mode**, in a similar way to what you have done in the previous Lab 2. You can test the function by adding the string `"scan -t <test no.>"` at the end of the command `lcom_run lab3` where the value of `<test no.>` must be one of 0 to 5. When `<test no.>` is **different** from 0, the LCF will behave deterministically, i.e., it will always generate the same sequence of scancodes, always terminating with the makecode and the breakcode of the ESC key.

```
minix$ lcom_run lab3 "scan -t <test no.>" # <test no.>: 0, 1, 2, 3, 4, or 5
```

Test **n° 1** just tests if your code **handles properly the ESC key**. Test **n° 5** issues a sequence of 5 keys (followed by the ESC key) and is the only test that checks whether your program computes correctly the **number of `sys_inb()` calls**.

The purpose of test **n° 0** is to **extensively test** your code. Thus, the LCF will generate a sequence of **random** makecode and breakcode pairs. Some of these codes may not correspond to any keyboard key. The **length** of this sequence is also **random**.

We suggest that you execute the tests starting with test **n° 1** and up to test **n° 5**. You should run test **n° 0** only after running and passing each of the other tests.

In test mode, the LCF checks if your program behaves as expected. In this mode, the LCF itself generates the scancodes, thus emulating the keyboard and the user that types on the keyboard. Please note that the LCF does not try to emulate a human user, instead the scancode bytes are “sent” in quick succession, thus its temporal behavior is not what you would observe if a human user was using the keyboard.

Before exiting, the LCF prints a message with the result of the test, either **“Test FAILED”** or **“Test succeeded”**. Furthermore, it prints the sequence of scancode bytes injected, after the line with the test outcome. The value of these bytes are represented in hexadecimal, but they are not prefixed with “0x”. This is intended to help debugging, if needed.

Next Steps

Congratulations for passing all the LCF tests! The next steps for this lab focus on learning how to interact with the KBC. However, it is important to recall the [Polling and Interrupt mechanisms](#)

Interrupts vs Polling

By now, you should be familiar with both polling and interrupts, especially interrupts, since it was used to develop the previous lab exercises. However, it is worthwhile to revisit both mechanisms and recall their key aspects.

💡 Tip

- Check out: [Polling](#)
- Check out: [Interrupts](#)

Interrupts or Polling, which mechanism should be used?

Let us compare the **Response time** and **Efficiency/Overhead** between both mechanisms:

Aspect	Polling	Interrupts
Response Time	Highly variable since the response time depends on the polling frequency and the tasks the CPU has to do between consecutive polls.	Usually responsive, as the CPU can respond almost immediately when a device raises an interrupt, depending on the interrupt priority.
CPU Efficiency / Overhead	Less per-check overhead, but inefficient overall when events are rare, because the CPU wastes cycles repeatedly checking for events that may not have occurred.	Higher per-event overhead, because the CPU must save context, execute the interrupt service routine (ISR), and restore state. Overall efficiency is higher since the CPU can perform other tasks while waiting for events.
Best Use Case	Simple or very high-speed devices where events occur constantly and are predictable.	Devices where events are sporadic or asynchronous, or when CPU time must be used efficiently for other tasks.

Why interrupts are still usually preferred?

Despite the overhead, interrupts are more efficient overall in most systems because the inputs on I/O devices are typically sporadic and the CPU does not waste cycles repeatedly checking (polling) a device that may have no new data. Interrupts allow the CPU to perform other tasks and respond only when an event occurs.

Next Steps

Now that interrupts and polling have been covered, let's dive deeper into the Keyboard Controller (KBC) and its commands in [Managing KBC](#).

Managing KBC

The Keyboard Controller (**KBC**), often referring to the classic **i8042** controller, takes **KBC Commands** to control its operation, therefore allowing you to enable or disable interrupts from the Keyboard or Mouse and managing the **KBD Interface**, the serial interface between the keyboard and the KBC.

KBC Commands

To be able to control the **KBC**, these commands must be written to the **Command Register** at address **0x64**. Depending of which command you issue, the **KBC** may need an additional **argument** or **return** a value. If an argument is required, this should be written to the **Input Buffer** at address **0x60**. Finally, if the command answers, the return value should be read from the **Output Buffer** at address **0x60**.

Specification

The following table shows the main KBC commands:

Command	Meaning	Arguments (Arg) / Return Value (R)
0x20	Read Command Byte	Returns Command Byte
0x60	Write Command Byte	Takes Command Byte as Arg
0xAA	Check KBC (Self-test)	Returns 0x55 if OK, Returns 0xFC if error
0xAB	Check Keyboard Interface	Returns 0 if OK
0xAD	Disable KBD Interface	
0xAE	Enable KBD Interface	

Note

There are several other KBC-commands related to the **mouse** (and also to the keyboard)

KBC Command Byte

The **KBC Command Byte** is a single 8-bit register inside the i8042 keyboard controller that contains the keyboard configuration and can be issued to change the behavior of the keyboard and mouse interface.

The following table presents the bit fields of the KBC Command Byte:

Bit	Name	Meaning (If set to 1)
7	-	-
6	-	-
5	DIS2	Disable mouse interface
4	DIS	Disable keyboard interface
3	-	-
2	-	-
1	INT2	Enable interrupts from mouse
0	INT	Enable interrupts from keyboard

Note

- Either not used or not relevant for Lab

Reading Command Byte

Let's say that you want to check if the **Keyboard interrupts are enabled**. This verification can be done by **reading** the **Command Byte**, which implies that the **KBC Command 0x20** is issued to the **KBC** enabling you to **read** the **Command Byte**, and checking if the bit in position 0 is set to 1.

Tip

1. Issue **KBC Command 0x20** to the **Command Register** at port **0x64**
2. Read the **KBC Command Byte** (return value) from **Output Buffer** at port **0x60**
3. Check the **KBC Command Byte** bits to, e.g., check if the Keyboard interrupts are enabled

Writing Command Byte

If you are required to change the **KBC** configuration, writing a new **Command Byte** is the way to do it. Therefore, the **KBC Command 0x60** must be issued, which takes as argument the new version of **KBC Command Byte**. Do not forget that any **KBC Command** should be written to the **Command Register** and the arguments should be passed to the **Input Buffer**.

Tip

1. Read the **KBC Command Byte** following the previous steps
2. Update the **KBC Command Byte** with the appropriated configuration values
3. Issue **KBC Command 0x60** to the **Command Register** at port **0x64**
4. Write the **KBC Command Byte** (argument) to the **Input Buffer** at port **0x60**

Issues Programming the Keyboard

The first issue is related to the **Output Buffer** multiple usages. As you may notice, the response values from the **KBC Commands** are available to read at the **Output Buffer**. Furthermore, if interrupts are enabled, the **IH** will always read the **Output Buffer** to check for new input.

Caution

These behaviours are not compatible since two behaviours can happen:

1. The **IH** can **"steal"** the return values from other code.
2. The **C@KBD** may send a **scancode** while you are writing a command to **KBC**
 - Instead of the command **return value**, the value read from the **Output Buffer** will be a **scancode**

(1) **When** to issue KBC Commands?

- Prevent **KBC** from generating interrupts.

(2) **How to prevent accepting a scancode** as a response to a command?*

- If all you need is to use **KBC** commands, then you can **disable** the **KBD** interface
- If you also need to issue **KBD** commands, then a complex solution is required
 - But in Lab 3, **we do not use KBD commands**.

The second issue is a **timing issue** since the **KBD/KBC** responses are not immediate. From the **IBM's** specification of the **i8042** controller, the keyboard must respond to a command

within **20 ms**. Thus your code cannot expect an immediate response after a command is issued.

⚠ Caution

Your code needs to **wait for long enough**, but **not indefinitely**

- Wait until the KBC reports a time-out.
- A fault-tolerant approach is to retry a few times on time-out, and finally give up

Because these time intervals are in the order of tens of **milliseconds**, the `sleep()` function is **not useful** since its resolution is **1 second**.

Instead, you can use the function `tickdelay()` of Minix 3's `libsys` as follows:

```
// minix/sysutil.h it is already included in <lcom/lcf.h>
#include <minix/sysutil.h>

#define DELAY_US    20000

tickdelay(micros_to_ticks(DELAY_US));
```

This function is similar to `sleep()` in that it suspends execution of the process that calls it for the time interval specified in its argument.

Issuing a Command to the KBC

To address this issues, you should be careful when writing a command to **KBC**. To better understand how to issue a command, we provide you the following snippet of code:

```
#define KBC_ST_REG 0x64
#define KBC_CMD_REG 0x64
#define KBC_ST_IBF BIT(1)

while ( 1 ) {
    sys_inb(KBC_ST_REG, &stat); /* assuming it returns OK */
    /* loop while 8042 input buffer is not empty */
    if( (stat & KBC_ST_IBF) == 0 ) {
        sys_outb(KBC_CMD_REG, cmd); /* no args command */
        return 0;
    }
    tickdelay(micros_to_ticks(DELAY_US));
}
```

i Note

- Cannot write to the address **0x64** while the **input buffer is full**

- Code leaves the loop only after trying *RETRIES* times or when it succeeds to output the data to the address **0x64**
 - To make your code resilient to failures in the KBC/keyboard, you should update the code to exit the loop after **N retries**.

Reading KBC Command Response

In the same way, we also provide you the following snippet of code for your better understanding of how to read a **KBC command response**.

```
#define KBC_OUT_BUF 0x60
#define KBC_PARITY_ERR BIT(7)
#define KBC_TIMEOUT_ERR BIT(6)
#define KBC_OBF BIT(0)

while( 1 ) {
    sys_inb(KBC_ST_REG, &stat); /* assuming it returns OK */
    /* loop while 8042 output buffer is empty */
    if( stat & KBC_OBF ) {
        sys_inb(KBC_OUT_BUF, &data); /* ass. it returns OK */
        if ( (stat & (KBC_PARITY_ERR | KBC_TIMEOUT_ERR)) == 0 )
            return data;
        else
            return -1;
    }
    tickdelay(micros_to_ticks(DELAY_US));
}
```

Note

- Code leaves the loop only upon some input from the KBC_OUT_BUF
 - It is not robust against failures in the KBC/keyboard; you should modify it accordingly

Next Steps

Now that you are able to manage KBC and change its behaviour, the next task assigned to you is for you is [Reading Scancodes from KBC using Polling](#). Keep up the good work!

Reading Scancodes from KBC using Polling

The `kbd_test_poll()` function

The `kbd_test_poll()` function has the same goal as the previous `int kbd_test_scan()` function: it must correctly read all scancodes from the KBC. However, in this case, the use of interrupts is **forbidden**, and keyboard input must be handled using a **polling mechanism**. Do not forget to print the scancode on the console by calling the following provided function:

```
#include <lcom/lcf.h>

int kbd_print_scancode(bool make, uint8_t size, uint8_t bytes[]);
```

Where:

- **make:** indicates whether this is a makecode or a breakcode
- **size:** number of bytes of the scancode, 1 or 2
- **bytes:** is an array containing the bytes of the scancode read from the KBC. The array should contain the bytes **in the order they are read** from the KBC.

Note

Be aware that this function only accepts scancodes with 1 or 2 bytes, not supporting scancodes with 3 or more bytes (e.g., PAUSE, etc).

The output of this function will look like the following:

```
Makecode: 0x12
Breakcode: 0x92
Makecode: 0xe0 0x49
Breakcode: 0xe0 0xc9
```

Finally, this function must terminate when the breakcode of the ESC key, **0x81**, is detected.

Polling in MINIX

As said before, MINIX has a default **KBC IH** which is enabled when the OS boots. Nevertheless, calling the `lcf_start()` will automatically disable the KBC keyboard interrupts, also preventing the default **IH** from minix.

When your polling routine terminates, you **must enable** the KBC interrupts. If the interrupts are not re-enabled, the KBC will not generate further interrupts. This happens since `lcf_start()` disables keyboard interrupts before calling `kbd_test_poll()`, preventing Minix's keyboard IH from stealing the scancodes from your program.

💡 Tip

If the keyboard gets **stuck**, you should fix your code and access a remote shell via **ssh** to compile run it. The `lcf_start()` will **always reset** the KBC to a proper state, even if `kbd_test_poll()` does not enable interrupts correctly. Restarting the MINIX-LCOM VM also resets the KBC to the default state.

Enabling KBC interrupts

In the previous chapter, you have learned how to issue **KBC Commands**. This will be required to enable the Keyboard interrupts by **writing an appropriate KBC Command Byte**.

💡 Tip

- Read the current Command Byte
- Set the **INT** bit and write a new Command Byte
- Check out: [KBC Command Byte](#)
- Check out: [Writing Command Byte](#)

Running `kbd_test_poll()` function

After compiling using the `make` tool, you can execute the `kbd_test_poll()` function using the **lcom_run utility** as follows:

```
minix$ lcom_run lab3 "poll"
```

Testing with the LCF

Finally, you should test the `kbd_test_poll()` function using the **LCF testing mode** by adding the string **"-t <test no.>"** at the end of the command line where the value of **<test no.>** must be one of 0 to 5. When **<test no.>** is **different** from 0, the LCF will behave deterministically, i.e., it will always generate the same sequence of scancodes, always terminating with the makecode and the breakcode of the ESC key.

```
minix$ lcom_run lab3 "poll -t <test no.>" # <test no.>: 0, 1, 2, 3, 4, or 5
```

Since the `kbd_test_poll()` should have the same behaviour of `kbd_test_scan()`, the LCF tests are the same for both functions. Therefore, you can check the test specifications in the previous explanation of [Testing with the LCF](#).

Again, we suggest that you execute the tests starting with test **nº 1** and up to **nº 5**, and executing test **nº 0** only after passing each of the other tests.

Note

In test mode, the LCF checks if your program behaves as expected. In this mode, the LCF itself generates the scancodes, thus emulating the keyboard and the user that types on the keyboard. Please note that the LCF does not try to emulate a human user, instead the scancode bytes are “sent” in quick succession, thus its temporal behavior is not what you would observe if a human user was using the keyboard.

Before exiting, the LCF prints a message with the result of the test, either **“Test FAILED”** or **“Test succeeded”**. Furthermore, it prints the sequence of scancode bytes injected, after the line with the test outcome. The value of these bytes are represented in hexadecimal, but they are not prefixed with “0x”. This is intended to help debugging, if needed.

Question: `sys_inb()` Kernel Calls

When comparing `kbd_test_poll()` and `kbd_test_scan()` functions, will there be any difference on the number of `sys_inb()` kernel calls? If so, can you explain the reason why?

Tip

Increment a counter when calling `sys_inb()` kernel call and print it at the end of both functions.

Since you do not want this counter in your final project, you can take advantage of `#ifdef` for conditional compilation, which allows you to **exclude** the counter code from the final build.

```
#define LAB3
sys_inb(...);
#ifdef LAB3
counter++;
#endif
```

Finally, do not forget to add `CFLAGS += -DLAB3` to the **Makefile**

Next Steps

Congratulations for passing all the tests and finishing both functions!! In the next class, your task will require the usage of both the PC Keyboard and the PC Timer. Therefore, be ready to [Handle multiple I/O Devices](#).

Handling multiple I/O Devices

Finally, you have reached the final lab task where you will develop a function capable of handling two I/O devices: the **PC Timer** and the **PC Keyboard**.

The `kbd_test_timed_scan(uint8_t n)` function

Yet again, this function is very similar to the `kbd_test_scan()` in the sense that it should subscribe the **keyboard interrupts** and **print the scancodes** received from the keyboard, by invoking `kbd_print_scancode()`.

The **key** difference is that this function receives the argument `uint8_t n`, referring to a maximum idle interval. Therefore, your function has two termination conditions:

-
- When the user **releases the ESC key**
 - When no scancode is received for the **number of seconds equal to its argument**
-

In order to measure the time interval, you **must** use the interrupts of the **PC's Timer 0**. Furthermore, the configuration of PC's Timer 0 **does not need** to be modified, so you only need to **subscribe** the **Timer 0** interrupts, as done in `timer_test_int()` of Lab 2.

Including the Timer Lib

The last step of the previous lab required you to build and include a reusable library with the main Timer functions. Since you should take advantage of the functions developed during Lab 2, e.g. `timer_int_handler()`, the Timer Lib should be included in our program.

To achieve this, we must update our **Makefile** to specify the required **library path**, allowing the compiler and linker to correctly **incorporate the library** into our program.

```
CFLAGS += -I../lab2          # Specify directories to the compiler to search for
header files                  # This is not needed if you included them in the
                               `libtimers.a` archive

LDADD += -llcf -L../lab2     # Tell the compiler to look in the lab2 directory for
libraries                    # Link the library named libtimers.a
LDADD += -ltimer
```

Running `kbd_test_timed_scan()` function

After compiling using the `make` tool, you can execute the `kbd_test_timed_scan()` function using the **lcom_run utility** as follows:

```
minix$ lcom_run lab3 "timed <decimal no. - time>"
```

Testing with the LCF

Finally, you should test the `kbd_test_timed_scan()` function using the **LCF testing mode** by adding the string `"-t <test no.>"` at the end of the command line where the value of `<test no.>` must be one of 0 to 10. When `<test no.>` is **different** from 0, the LCF will behave deterministically, i.e., it will always generate the same sequence of scancodes, always terminating with the makecode and the breakcode of the ESC key.

```
minix$ lcom_run lab3 "timed <decimal no. - time> -t <test no.>"
```

Test **n° 1** to **n° 5** are similar to those of `kbd_test_scan()` or `kbd_test_poll()` since they inject exactly the same scancodes and your program is expected to terminate upon reading the breakcode of the ESC key.

Test **n° 6** to **n° 10** verify whether your program terminates when the keyboard is idle for the time interval specified. These tests inject exactly the same sequences except for the last two scancodes, the ESC key's make and break codes, which are never injected.

Again, we suggest that you execute the tests starting with test **n° 1** and up to **n° 10**, and executing test **n° 0** only after passing each of the other tests.

Note

In test mode, the LCF checks if your program behaves as expected. In this mode, the LCF itself generates the scancodes, thus emulating the keyboard and the user that types on the keyboard. Please note that the LCF does not try to emulate a human user, instead the scancode bytes are "sent" in quick succession, thus its temporal behavior is not what you would observe if a human user was using the keyboard.

Before exiting, the LCF prints a message with the result of the test, either **"Test FAILED"** or **"Test succeeded"**. Furthermore, it prints the sequence of scancode bytes injected, after the line with the test outcome. The value of these bytes are represented in hexadecimal, but they are not prefixed with "0x". This is intended to help debugging, if needed.

Final Notes

Well done! You have finished all functions of Lab 3.

Before jumping into Lab 4, where you will be introduced to the **PS/2 Mouse**, we recommend you to create a reusable library for the **PC Keyboard** as we did for the **PC Timer**.

Tip

- Check out: [Creating and Including the Timer Library](#)